❤️ Heart Disease Classification — End-to-End Project

Goal: predict whether a patient has heart disease (`target` = 1) or not (`target` = 0) from 13 clinical measurements — a **binary classification** problem.

Dataset: `heart-disease.csv` (Cleveland heart disease data; 303 patients, 13 features + target). All numeric, no missing values.

The workflow this notebook follows:

1. Setup & imports
2. Load the data & explore (EDA)
3. Features, target & train/test split
4. Baseline comparison of 9 models
5. Scaling (fix distance-based models)
6. Cross-validation (a trustworthy ranking)
7. Hyperparameter tuning (by hand → RandomizedSearchCV → GridSearchCV)
8. Evaluation beyond accuracy (confusion matrix, precision/recall/F1, ROC-AUC, feature importance)
9. Tuning the actual leader (Logistic Regression)
10. Final model — interpret, save & predict on a new patient

Takeaway up front: on small data, trust **cross-validation** over a single train/test split, and pick metrics that match the goal (for disease detection, **recall** matters most).

---

1. Setup & Imports

Standard scientific-Python stack: `numpy` (math), `pandas` (tables), `matplotlib` (plots).

In [1]:
```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

2. Load the Data & Explore (EDA)

Before modeling, understand the data: shape → `head` → `info` (dtypes) → `isna().sum()` (missing values) → `describe` (ranges) → target balance → correlations with the target.

Why this matters: EDA dictates the whole strategy. Here the data is *all numeric with no missing values* (rare and lucky), the target is *roughly balanced* (~165 vs ~138, so accuracy is a fair metric), and features like `cp` (chest pain) and `thalach` (max heart rate) correlate most with disease.

In [2]:
```
df = pd.read_csv("/Users/nishantsoni/Desktop/personal/ML project/hands on projects by me/supervisec
df.shape
```

Out[2]:
```
(303, 14)
```

In [3]:
```
df.head()
```

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       303 non-null    int64  
 1   sex       303 non-null    int64  
 2   cp        303 non-null    int64  
 3   trestbps  303 non-null    int64  
 4   chol      303 non-null    int64  
 5   fbs       303 non-null    int64  
 6   restecg   303 non-null    int64  
 7   thalach   303 non-null    int64  
 8   exang     303 non-null    int64  
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64  
 11  ca        303 non-null    int64  
 12  thal      303 non-null    int64  
 13  target    303 non-null    int64  
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

In [5]:
```
df.isna().sum()
```

Out[5]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [6]:
```
df.describe()
```

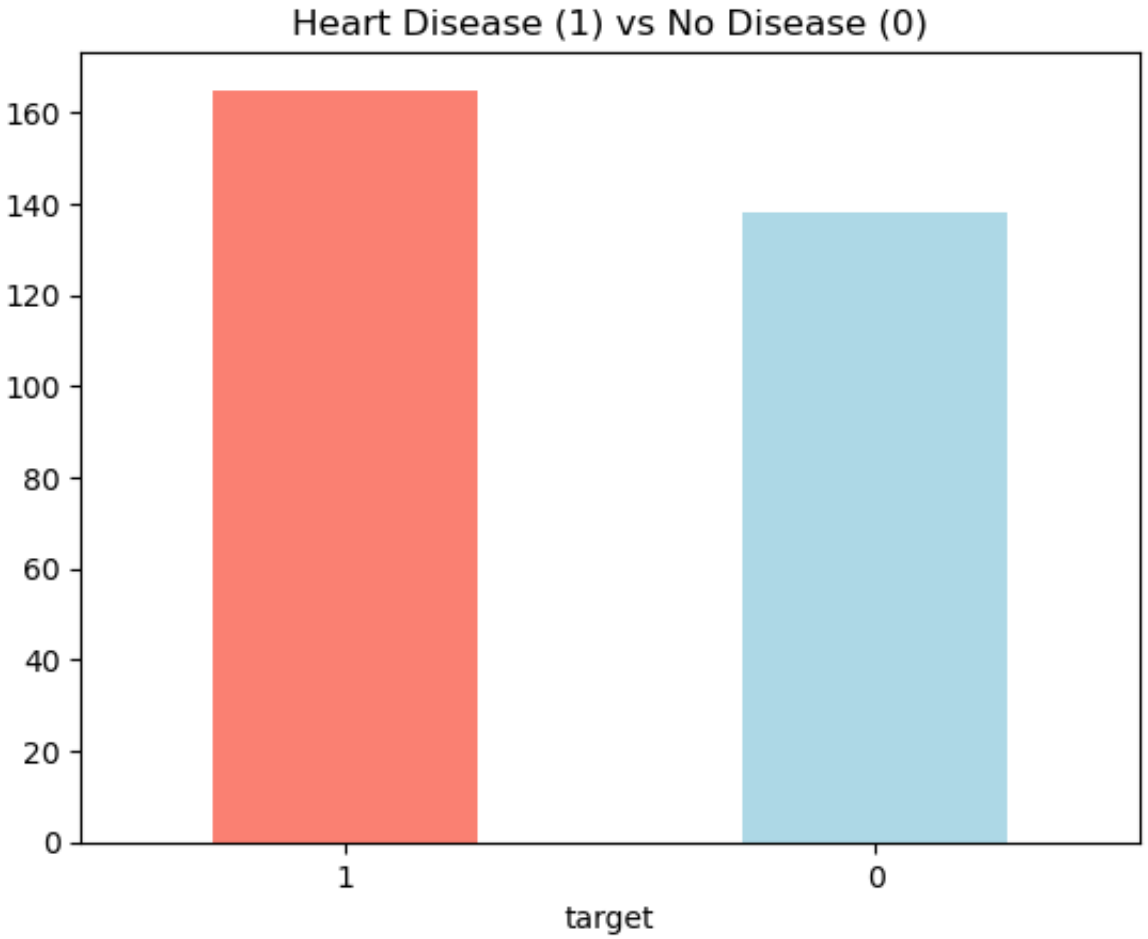
Out[6]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	3(
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	

In [7]:
```
df["target"].value_counts()
```

Out[7]:
```
target
1    165
0    138
Name: count, dtype: int64
```

In [8]:
```
df["target"].value_counts().plot(kind="bar", color=["salmon", "lightblue"])
plt.title("Heart Disease (1) vs No Disease (0)")
plt.xticks(rotation=0);
```



In [9]:
```
df.corr()["target"].sort_values(ascending=False)
```

Out[9]:
```
target      1.000000
cp          0.433798
thalach     0.421741
slope       0.345877
restecg     0.137230
fbs        -0.028046
chol       -0.085239
trestbps   -0.144931
age        -0.225439
sex        -0.280937
thal       -0.344029
ca         -0.391724
oldpeak    -0.430696
exang      -0.436757
Name: target, dtype: float64
```

3. Features (X), Target (y) & Train/Test Split

Split the table into **X** (the 13 input columns) and **y** (the `target` answer), then hold out 20% as a **test set** the models never see during training.

- `random_state=42` → reproducible split (same rows every run).
- `stratify=y` → keeps the disease/no-disease ratio identical in train and test (good practice for classification).

Golden rule: the test set is *sacred* — touch it only for the final score, never for training or tuning.

In [10]:
```
X = df.drop("target", axis=1)   # everything EXCEPT the answer
y = df["target"]                # just the answer
X.shape, y.shape
```

Out[10]:
```
((303, 13), (303,))
```

In [11]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
X_train.shape, X_test.shape
```

Out[11]: ((242, 13), (61, 13))

4. Baseline Model Comparison

Try many model *families* with default settings (no scaling, no tuning) to get a starting point. Putting models in a **dictionary** lets us loop and treat them identically.

What to expect: tree-based models (RandomForest, GradientBoosting, XGBoost, CatBoost) do well out-of-the-box because they're *scale-invariant*. KNN and SVM look weak here — **not** because they're bad, but because we fed them *unscaled* data (they rely on distances). That's the lesson that motivates Section 5.

> XGBoost and CatBoost are external libraries: `pip install xgboost catboost` (and on Mac, `brew install libomp` if XGBoost won't import).

In [12]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
```

In [14]:
```
models = {
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "KNN":                KNeighborsClassifier(),
    "SVM":                SVC(),
    "DecisionTree":       DecisionTreeClassifier(random_state=42),
    "RandomForest":       RandomForestClassifier(random_state=42),
    "GradientBoosting":   GradientBoostingClassifier(random_state=42),
    "XGBoost":            XGBClassifier(random_state=42),
    "CatBoost":           CatBoostClassifier(random_state=42, verbose=0),
```

```
    "NaiveBayes":          GaussianNB(),
}
```

In [15]:
```
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)                    # train on training data
    results[name] = model.score(X_test, y_test)    # accuracy on unseen test data
results
```

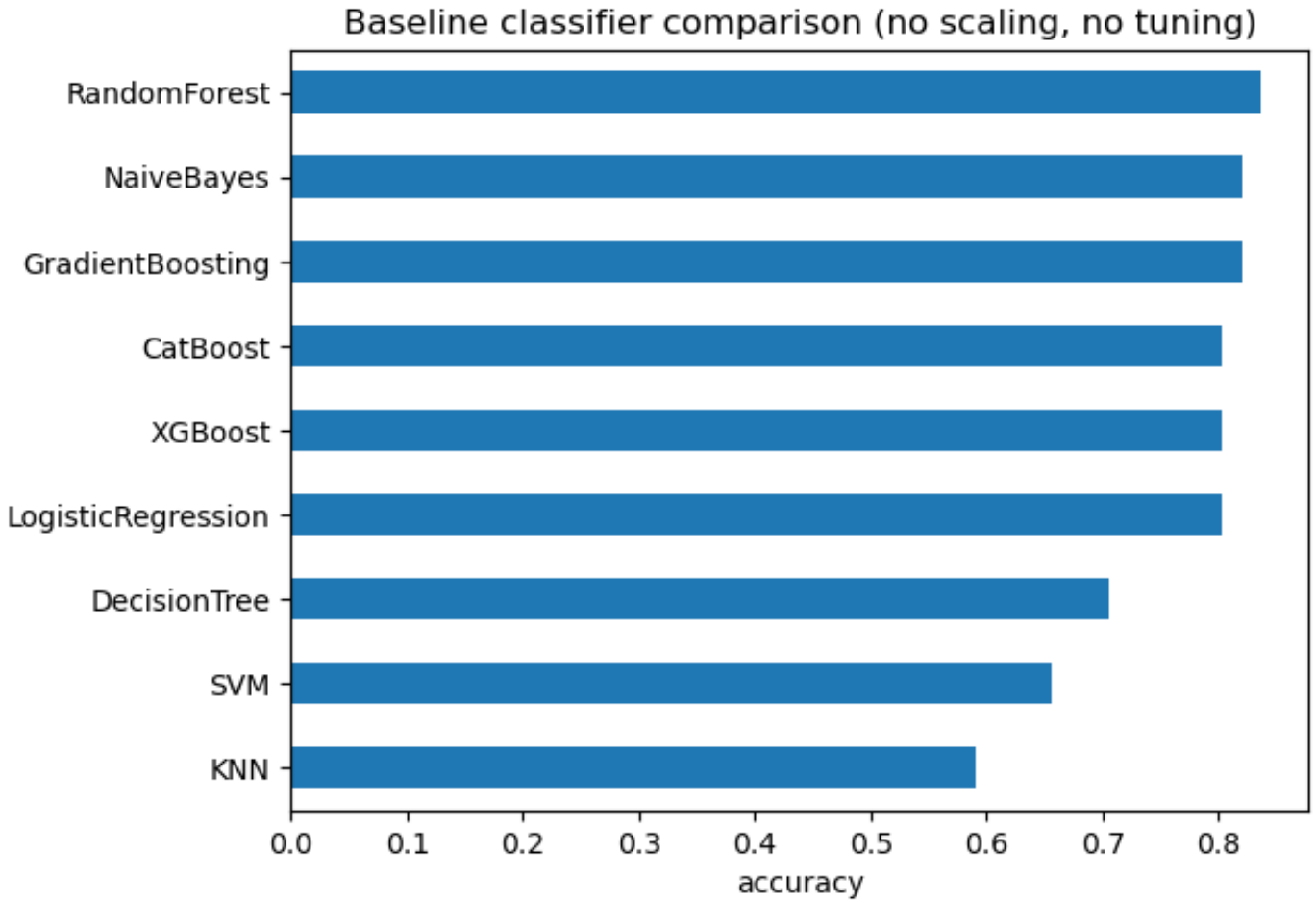
Out[15]:
```
{'LogisticRegression': 0.8032786885245902,
 'KNN': 0.5901639344262295,
 'SVM': 0.6557377049180327,
 'DecisionTree': 0.7049180327868853,
 'RandomForest': 0.8360655737704918,
 'GradientBoosting': 0.819672131147541,
 'XGBoost': 0.8032786885245902,
 'CatBoost': 0.8032786885245902,
 'NaiveBayes': 0.819672131147541}
```

In [16]:
```
results_df = pd.DataFrame(results, index=["accuracy"]).T
results_df.sort_values("accuracy", ascending=False)
```

Out[16]:

	accuracy
RandomForest	0.836066
GradientBoosting	0.819672
NaiveBayes	0.819672
LogisticRegression	0.803279
XGBoost	0.803279
CatBoost	0.803279
DecisionTree	0.704918
SVM	0.655738
KNN	0.590164

In [17]:
```
results_df.sort_values("accuracy").plot(kind="barh", legend=False)
plt.xlabel("accuracy")
plt.title("Baseline classifier comparison (no scaling, no tuning)");
```



📝 Note: why do several models show the *exact* same accuracy?

It's a coincidence forced by the **small test set**, not because the models are identical.

The test set has only **61 patients**, so accuracy can only land on steps of 1/61 ≈ 0.0164. Check the math:

- `0.8033 × 61 = 49` → those models each got **49 of 61** correct.
- `0.8197 × 61 = 50` → those each got **50 of 61** correct.

With only ~62 possible accuracy values, ties are common. And **same accuracy ≠ same predictions** — two models

can each get 49 right while disagreeing on *which* patients. The next cells confirm they disagree on individual patients and have different predicted probabilities.

In [18]:
```
lr  = LogisticRegression(max_iter=1000).fit(X_train, y_train)
xgb = XGBClassifier(random_state=42).fit(X_train, y_train)

lr_pred  = lr.predict(X_test)
xgb_pred = xgb.predict(X_test)

# how many of the 61 predictions are identical?
(lr_pred == xgb_pred).sum(), len(y_test)
```

Out[18]:
```
(55, 61)
```

In [19]:
```
lr.predict_proba(X_test)[:5].round(3)
```

Out[19]:
```
array([[0.898, 0.102],
       [0.816, 0.184],
       [0.993, 0.007],
       [0.284, 0.716],
       [0.314, 0.686]])
```

In [20]:
```
xgb.predict_proba(X_test)[:5].round(3)
```

Out[20]:
```
array([[0.981, 0.019],
       [0.301, 0.699],
       [1.   , 0.   ],
       [0.023, 0.977],
       [0.128, 0.872]], dtype=float32)
```

5. Scaling — Fix the Distance-Based Models

KNN and SVM measure distances, so a large-range feature (`chol` , in the hundreds) drowns out a small one (`oldpeak` , near 0–6). **Standardizing** (mean 0, std 1) puts every feature on equal footing.

We scale the *right* way — inside a `Pipeline` via `make_pipeline(StandardScaler(), model)` — so the

scaler is fit on training data only (no leakage). Tree models don't need scaling (they only care about value *order*), so we leave them raw.

Watch: KNN and especially SVM jump up after scaling. A "bad" score was really a *preparation* problem, not a model problem.

In [22]:
```
# Now the two upgrades that fix what we saw: scaling (to rescue KNN and SVM) and cross-validation (
# Chunk 24 — prove that scaling rescues KNN
# First, see the effect directly. We compare unscaled KNN (what you had) with a scaled version. mak

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

knn_raw = KNeighborsClassifier().fit(X_train, y_train)
print("KNN unscaled:", round(knn_raw.score(X_test, y_test), 4))

knn_scaled = make_pipeline(StandardScaler(), KNeighborsClassifier()).fit(X_train, y_train)
print("KNN scaled  :", round(knn_scaled.score(X_test, y_test), 4))

## You should see a clear jump — same model, same data, just put on a fair scale.
```

```
KNN unscaled: 0.5902
KNN scaled  : 0.8033
```

In [24]:
```
# Chunk 25 — same story for SVM
svm_raw = SVC().fit(X_train, y_train)
print("SVM unscaled:", round(svm_raw.score(X_test, y_test), 4))

svm_scaled = make_pipeline(StandardScaler(), SVC()).fit(X_train, y_train)
print("SVM scaled  :", round(svm_scaled.score(X_test, y_test), 4))

# SVM usually jumps the most — its previous weak score was entirely a scaling problem, not the mode
```

```
SVM unscaled: 0.6557
SVM scaled  : 0.8197
```

In [25]:
```
# Chunk 26 — rebuild the dictionary, scaling only the models that need it
# The three distance/linear models get wrapped in a scaling pipeline; the tree-based models stay ra
models = {
```

```
    "LogisticRegression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "KNN":                make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "SVM":                make_pipeline(StandardScaler(), SVC()),
    "DecisionTree":       DecisionTreeClassifier(random_state=42),
    "RandomForest":       RandomForestClassifier(random_state=42),
    "GradientBoosting":   GradientBoostingClassifier(random_state=42),
    "XGBoost":            XGBClassifier(random_state=42),
    "CatBoost":           CatBoostClassifier(random_state=42, verbose=0),
    "NaiveBayes":         GaussianNB(),
}
```

6. Cross-Validation — A Trustworthy Ranking

Instead of one lucky 61-patient split, `cross_val_score` splits the data into 5 folds, scores each in turn, and averages. Because each pipeline is refit inside every fold, the scaler only ever learns from that fold's training portion — **no leakage** (this is why we wrapped scaling in a Pipeline).

Result: the baseline ties resolve into a real ranking, and the scores are now trustworthy. The `+/-` is the std across folds — a measure of *stability*, not just accuracy.

In [26]:
```
# Chunk 27 — cross-validation instead of one split
# This is the big one. Instead of scoring on a single 61-patient test set, cross_val_score splits t
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = {}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    cv_results[name] = scores.mean()
    print(f"{name:20s}: {scores.mean():.4f}  (+/- {scores.std():.4f})")
```

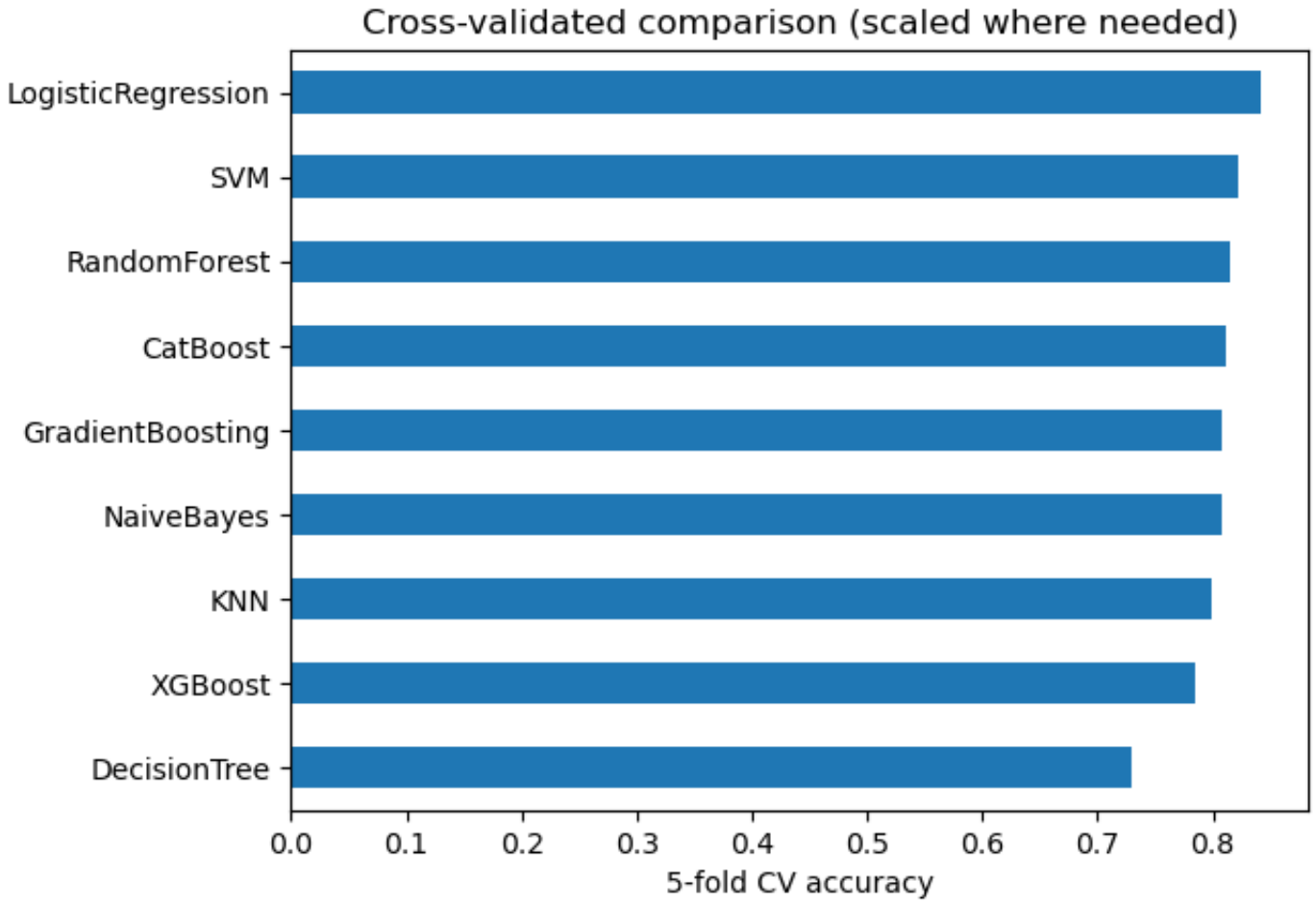
```
LogisticRegression  : 0.8416  (+/- 0.0448)
KNN                 : 0.7985  (+/- 0.0714)
SVM                 : 0.8217  (+/- 0.0534)
DecisionTree        : 0.7289  (+/- 0.0655)
RandomForest        : 0.8152  (+/- 0.0334)
GradientBoosting    : 0.8087  (+/- 0.0513)
XGBoost             : 0.7856  (+/- 0.0335)
CatBoost            : 0.8118  (+/- 0.0527)
NaiveBayes          : 0.8084  (+/- 0.0697)
```

In [27]:
```
# Chunk 28 — sorted results table
cv_df = pd.DataFrame(cv_results, index=["cv_accuracy"]).T
cv_df.sort_values("cv_accuracy", ascending=False)
```

Out[27]:

	cv_accuracy
LogisticRegression	0.841585
SVM	0.821749
RandomForest	0.815191
CatBoost	0.811803
GradientBoosting	0.808689
NaiveBayes	0.808415
KNN	0.798470
XGBoost	0.785628
DecisionTree	0.728852

In [28]:
```
## Chunk 29 — plot the trustworthy ranking
cv_df.sort_values("cv_accuracy").plot(kind="barh", legend=False)
plt.xlabel("5-fold CV accuracy")
plt.title("Cross-validated comparison (scaled where needed)");
```



📝 Note: when to use `train_test_split` vs `cross_val_score`?

They're **not either/or** — they do different jobs and are used together.

- **`train_test_split`** makes *one* split → a held-out test set for the final honest check. Fast, but the score depends on which rows landed in test (unreliable on small data).
- **`cross_val_score`** *rotates* the split → every row is validated once; averaging gives a stable estimate (+ std). But it doesn't leave a single untouched "final exam" set by itself.

The professional pattern (uses both):

1. `train_test_split` → lock away a final test set. Don't touch it.
2. `cross_val_score` / CV inside GridSearchCV on the *training* part → compare models & tune. *All decisions happen here.*
3. Score the chosen, tuned model **once** on the held-out test set → the honest final number.

Use `train_test_split`	Use `cross_val_score`
Final hold-out check	Comparing models / tuning
Big data (one split reliable)	Small data (one split unreliable — *our case*)
Slow models / deep learning	Want a stable estimate + uncertainty

7. Hyperparameter Tuning

Hyperparameters are settings *you* choose before training (e.g. KNN's `n_neighbors`, RandomForest's `n_estimators`). Tuning = searching for the values that give the best **cross-validated** score — done on the *training* data, with the test set kept sacred.

7a. By hand — tuning KNN's `k`

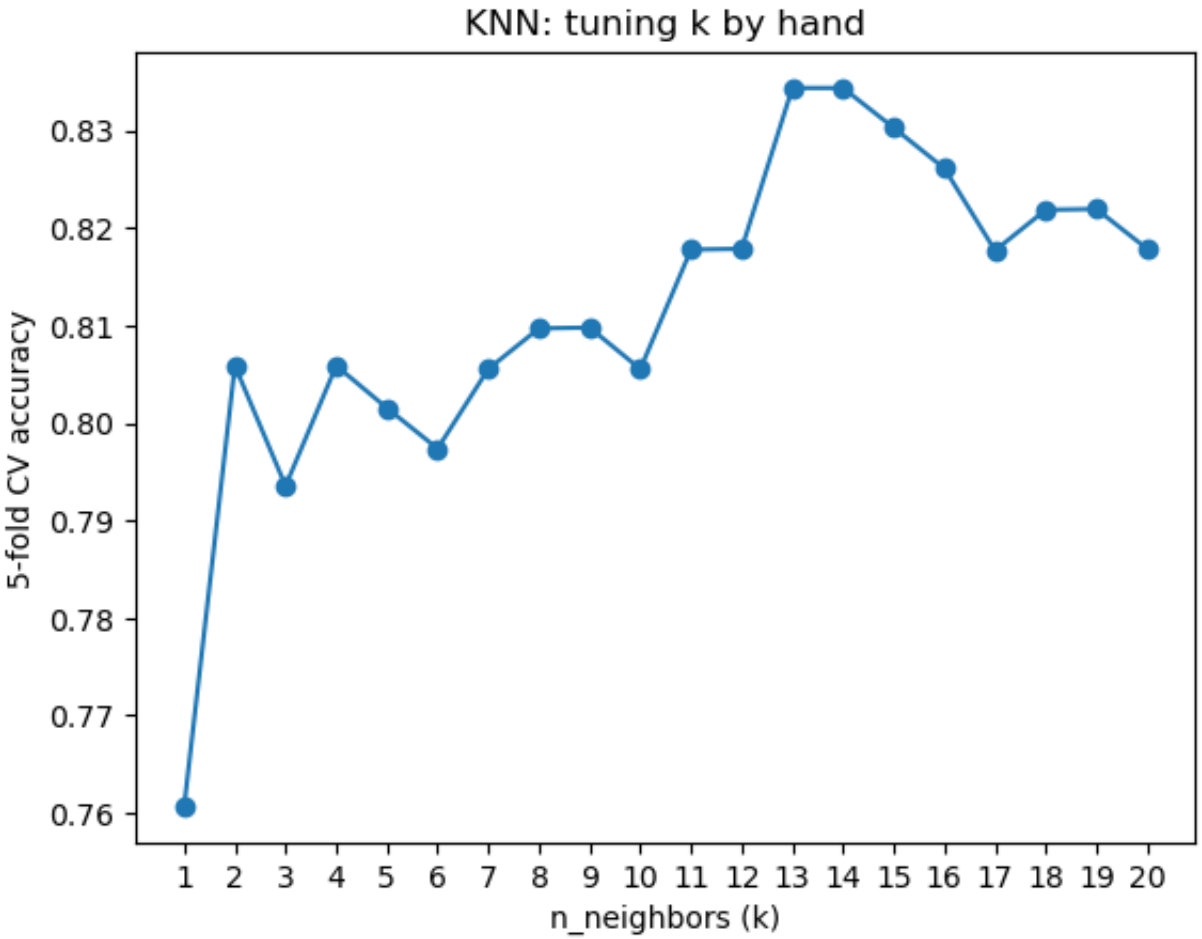
Loop over `k = 1…20`, CV-score each, and plot the curve. Small `k` overfits (jagged), large `k` underfits (too smooth); the peak is the sweet spot. By-hand works for *one* knob — for several, we automate below.

In [29]:
```
# Tuning — Part 1: by hand, then automated
# Now your favorite part. We tune on the training data with CV, keeping X_test untouched for the fi
# Chunk 30 — tune KNN's n_neighbors by hand
# The clearest way to see what tuning is: try every k and CV-score each. (We score on X_train only

neighbors = range(1, 21)
knn_scores = []
```

```
for k in neighbors:
    knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    score = cross_val_score(knn, X_train, y_train, cv=cv, scoring="accuracy").mean()
    knn_scores.append(score)

knn_scores
```

Out[29]:
```
[0.7607142857142857,
 0.8058673469387756,
 0.7935374149659864,
 0.8058673469387756,
 0.801530612244898,
 0.7973639455782313,
 0.8055272108843538,
 0.8096938775510203,
 0.8097789115646259,
 0.8055272108843538,
 0.8177721088435375,
 0.8178571428571428,
 0.8343537414965987,
 0.8343537414965987,
 0.8302721088435374,
 0.8261054421768707,
 0.8176870748299321,
 0.8218537414965986,
 0.8219387755102041,
 0.8178571428571428]
```

In [30]:
```
## Chunk 31 — plot the k curve
plt.plot(neighbors, knn_scores, marker="o")
plt.xlabel("n_neighbors (k)")
plt.ylabel("5-fold CV accuracy")
plt.title("KNN: tuning k by hand")
plt.xticks(neighbors);
```



In [31]:
```
## Chunk 32 — grab the best k
best_k = neighbors[np.argmax(knn_scores)]
print("Best k:", best_k, "| CV accuracy:", round(max(knn_scores), 4))

# That's the whole idea of tuning: search hyperparameter values for the best CV score.
# By-hand works for one knob — but most models have several, and trying all combos by hand is hopel
# That's what RandomizedSearchCV automates.
```

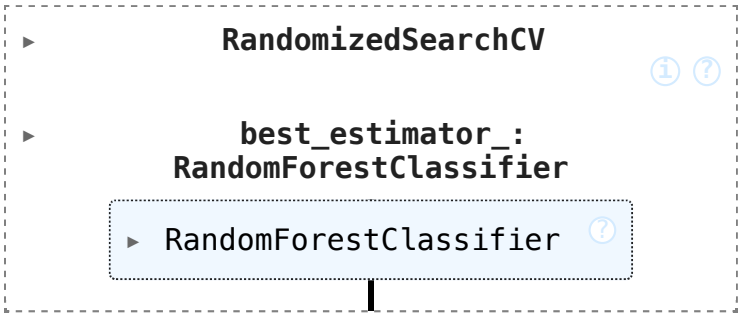
```
Best k: 13 | CV accuracy: 0.8344
```

7b. RandomizedSearchCV — for models with *many* knobs

RandomForest has many hyperparameters, so trying every combination is infeasible. RandomizedSearchCV samples `n_iter` random combinations and CV-scores each — efficient for large search spaces. We tune on `X_train`; CV happens inside.

In [32]:
```
# Chunk 33 — RandomizedSearchCV on RandomForest
# RandomForest has many knobs. Instead of looping by hand, we define ranges and let it sample n_ite
from sklearn.model_selection import RandomizedSearchCV

rf_grid = {
    "n_estimators":      [100, 200, 300, 500],
    "max_depth":         [None, 5, 10, 20],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf":  [1, 2, 4],
    "max_features":      ["sqrt", "log2", None],
}

rs_rf = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    rf_grid, n_iter=20, cv=cv, scoring="accuracy",
    random_state=42, n_jobs=-1)

rs_rf.fit(X_train, y_train)   # tune on TRAINING data; CV happens inside
```

Out[32]:



In [35]:
```
# Chunk 34 — see what it found
print(rs_rf.best_params_)
round(rs_rf.best_score_, 4)   # best mean CV accuracy across the training folds
```

```
{'n_estimators': 300, 'min_samples_split': 10, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_
depth': None}
```

Out[35]: 0.8139

In [36]:
```
# Chunk 35 — the final honest score on the untouched test set
round(rs_rf.score(X_test, y_test), 4)   # the ONE time we use X_test
```

Out[36]: 0.8197

7c. GridSearchCV — zoom in around the best region

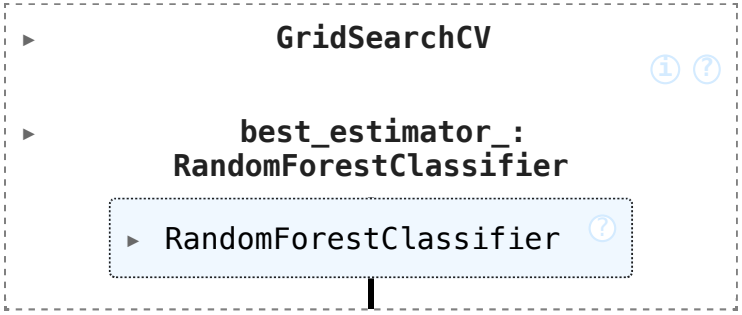
Now search *exhaustively* in a tight grid centered on what RandomizedSearch found, to refine. **Rule of thumb:** large space → RandomizedSearch; small space → GridSearch.

> On small, clean data, expect *diminishing returns* — GridSearch may barely beat (or even tie) RandomizedSearch. Knowing when you're near the performance ceiling is itself a skill.

In [38]:
```
# Tuning — Part 2: GridSearchCV (zoom in)
# The strategy from your notes: RandomizedSearchCV found a good region; GridSearchCV now searches e
# in a tight grid around those winning values to squeeze out a bit more.
# Chunk 36 — focused grid around the best params

from sklearn.model_selection import GridSearchCV

rf_grid2 = {
    "n_estimators":      [200, 300, 400],   # around 300
    "min_samples_split": [8, 10, 12],       # around 10
    "min_samples_leaf":  [3, 4, 5],          # around 4
    "max_features":      ["log2", "sqrt"],
    "max_depth":         [None, 10],
}

gs_rf = GridSearchCV(
    RandomForestClassifier(random_state=42),
    rf_grid2, cv=cv, scoring="accuracy", n_jobs=-1)

gs_rf.fit(X_train, y_train)
```

```
# Notice the values are centered on what RandomizedSearch found.
# This is 108 combos × 5 folds = 540 fits, so it'll take a few seconds.
# (Unlike RandomizedSearch, GridSearch tries every combination — fine for a small focused grid, imp
```

Out[38]:



In [39]:
```
gs_rf.best_params_
```

Out[39]:
```
{'max_depth': None,
 'max_features': 'log2',
 'min_samples_leaf': 5,
 'min_samples_split': 12,
 'n_estimators': 400}
```

In [40]:
```
round(gs_rf.best_score_, 4)
```

Out[40]:
```
0.8221
```

In [42]:
```
# Chunk 38 — final test score of the refined model
round(gs_rf.score(X_test, y_test), 4)
```

Out[42]:
```
0.8033
```

8. Evaluation — Beyond Accuracy

Accuracy hides *what kind* of errors a model makes — critical for a disease detector. We look at the **confusion matrix**, **precision/recall/F1** per class, and the **ROC curve + AUC**.

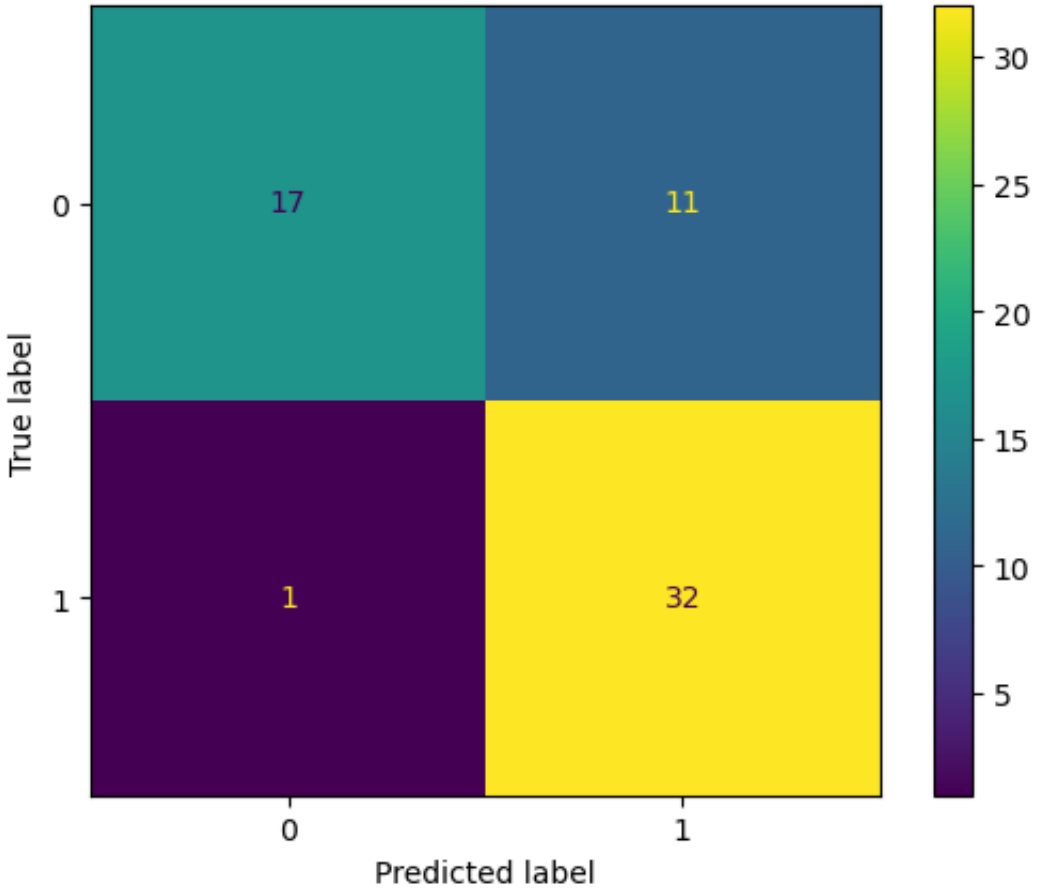
For disease screening, recall on class 1 matters most: a false negative (telling a sick patient they're healthy) is far

more dangerous than a false alarm. Also see **feature importances** — and check they line up with the EDA correlations from Section 2 (a sanity check that the model learned something real).

In [43]:
```
# Don't be surprised if GridSearch barely beats — or even ties — RandomizedSearch here. That's a ge
# not a failure: on a small, clean dataset like this (303 rows), models hit a performance ceiling q
# tuning gives diminishing returns. Knowing when you're near the ceiling (and to stop fiddling) is
# Whichever of the two scored highest is your champion — we'll evaluate gs_rf below (swap in rs_rf

# Evaluation — the full picture (beyond accuracy)
# Accuracy alone hides what kind of mistakes the model makes. For a disease detector, that matters
# Chunk 39 — predictions from the champion

y_pred = gs_rf.predict(X_test)
y_pred[:15]
```

Out[43]:
```
array([0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1])
```

In [44]:
```
# Chunk 40 — confusion matrix
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(gs_rf, X_test, y_test);
# Read it as a 2×2: top-left and bottom-right are correct; the off-diagonal are the two error types
# The bottom-left (false negatives — patients with disease the model said are healthy) is the dange
```



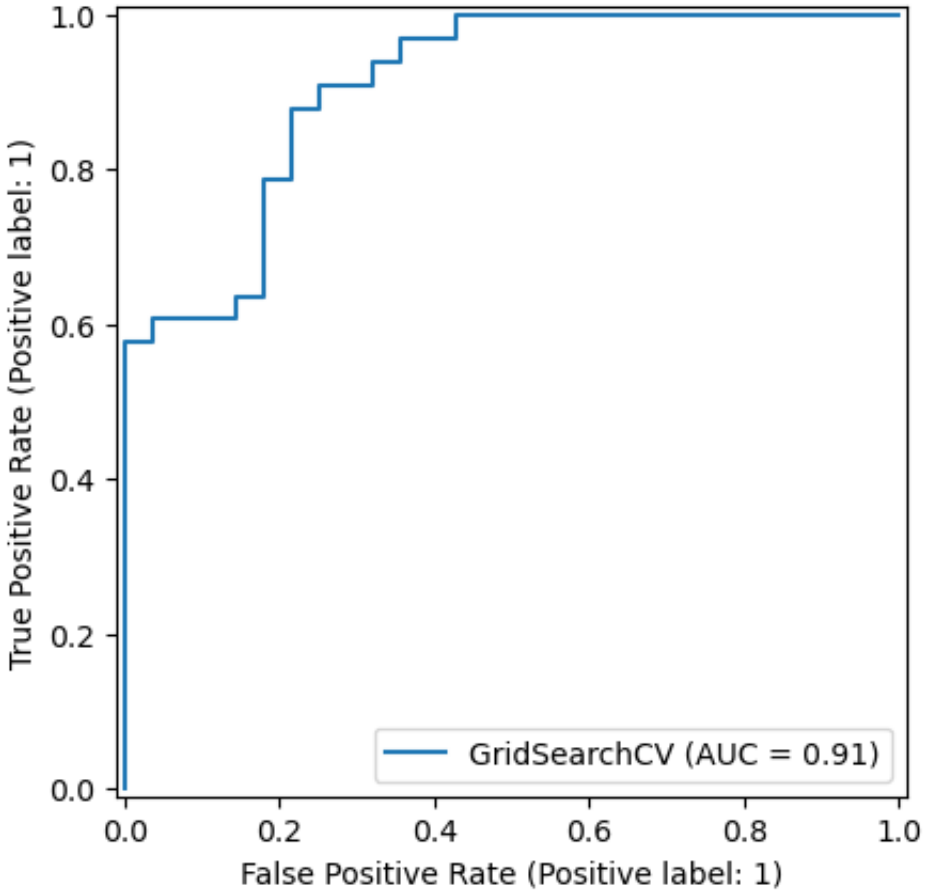
In [45]:
```
# Chunk 41 — precision, recall, F1 per class
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))

# For class 1 (has disease): precision = of those flagged as sick, how many really are; recall = of
# how many we caught. In a screening context you usually care most about recall for class 1 — missi
# worse than a false alarm.
```

```
              precision    recall  f1-score   support

           0       0.94      0.61      0.74        28
           1       0.74      0.97      0.84        33

    accuracy                           0.80        61
   macro avg       0.84      0.79      0.79        61
weighted avg       0.84      0.80      0.79        61
```

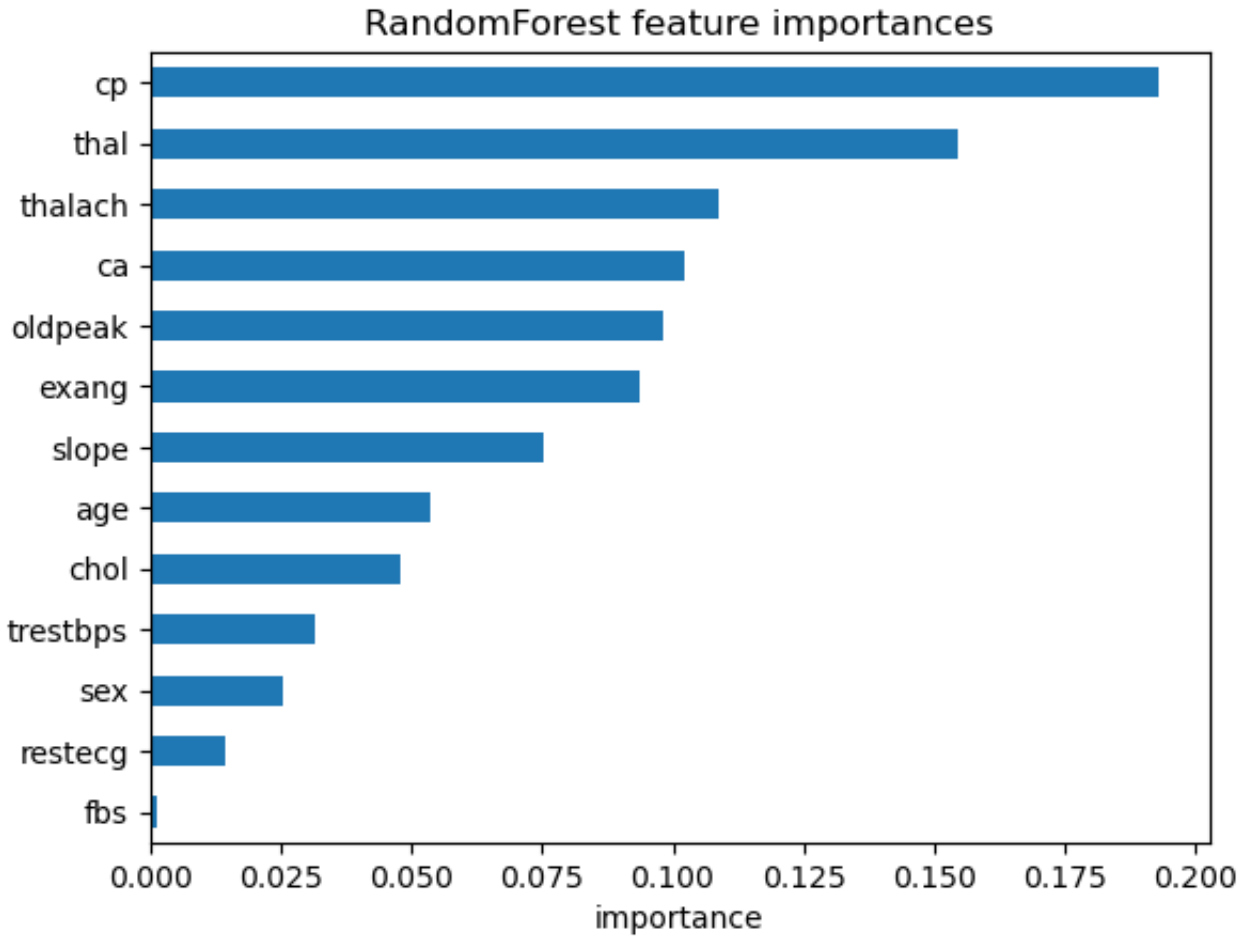
In [46]:
```python
# Chunk 42 — pull the key metrics out individually
from sklearn.metrics import precision_score, recall_score, f1_score

print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
print("F1       :", round(f1_score(y_test, y_pred), 4))

# These are the numbers you'd report. If recall is lower than you'd like for a medical use case, th
# the decision threshold — a topic for a future project.
```

```
Precision: 0.7442
Recall   : 0.9697
F1       : 0.8421
```

In [48]:
```python
# Chunk 43 — ROC curve and AUC
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(gs_rf, X_test, y_test);
# The ROC curve shows the trade-off between catching true positives and raising false alarms across
# AUC (shown in the legend) summarizes it — 1.0 is perfect, 0.5 is random guessing. AUC is threshol
# so it's a fairer single number than accuracy.
```



In [49]:
```
# Chunk 44 — which features drove the predictions?
importances = pd.Series(
    gs_rf.best_estimator_.feature_importances_, index=X.columns
).sort_values()

importances.plot(kind="barh")
plt.xlabel("importance")
plt.title("RandomForest feature importances");

# Compare this to the correlations you computed back in Chunk 9 — features like cp (chest pain), th
# and oldpeak should rank high in both. When the model's "reasoning" lines up with the exploratory
# check that it learned something real, not noise.
```



9. Tuning the Actual Leader — Logistic Regression

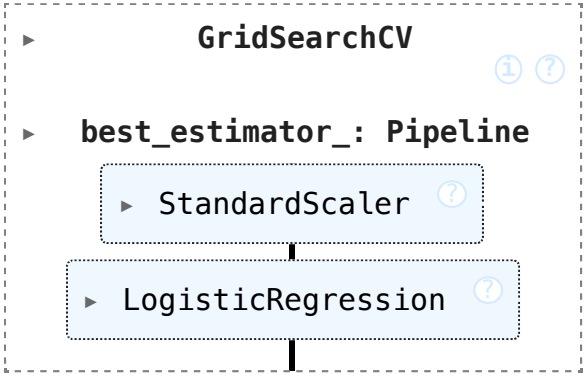
The cross-validated comparison (Section 6) showed **LogisticRegression is the best model (~0.84)**, with RandomForest third. So the *right* move is to tune the leader. (RandomForest was tuned above mainly as a teaching demo for RandomizedSearchCV, since it has many knobs; LogReg has few, so GridSearchCV fits it better.)

A subtle lesson from the RF numbers: RandomizedSearch gave CV 0.8139 / test 0.8197, while GridSearch gave a *higher* CV (0.8221) but *lower* test (0.8033). The higher-CV model scored lower on test — the noise of a 61-patient test set. Differences of 0.01–0.02 are within luck; lean on CV, don't chase the last decimal.

LogReg needs scaling, so we tune it *inside a pipeline* using the `step__param` naming (e.g. `model__C`). `C` is the main dial: small `C` = stronger regularization (simpler model).

In [51]:
```
# Chunk 45 — tune LogisticRegression (the real leader)


from sklearn.pipeline import Pipeline

logreg_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model",  LogisticRegression(max_iter=1000)),
])

logreg_grid = {
    "model__C":       [0.01, 0.1, 1, 10, 100],   # inverse regularization strength
    "model__penalty": ["l1", "l2"],               # l1 = sparse, l2 = smooth shrinkage
    "model__solver":  ["liblinear"],              # supports BOTH l1 and l2
}

gs_lr = GridSearchCV(logreg_pipe, logreg_grid, cv=cv, scoring="accuracy", n_jobs=-1)
gs_lr.fit(X_train, y_train)
```

Out[51]:



In [52]:
```
# Chunk 46 — best params and CV score
gs_lr.best_params_
```

Out[52]:
```
{'model__C': 0.01, 'model__penalty': 'l2', 'model__solver': 'liblinear'}
```

In [53]:
```
round(gs_lr.best_score_, 4)
```

Out[53]: 0.8299

In [54]:
```
# Chunk 47 — test score
round(gs_lr.score(X_test, y_test), 4)
```

Out[54]: 0.7869

In [55]:
```
# Chunk 48 — full metrics, to compare head-to-head with RF
y_pred_lr = gs_lr.predict(X_test)
print(classification_report(y_test, y_pred_lr))
```

```
              precision    recall  f1-score   support

           0       0.86      0.64      0.73        28
           1       0.75      0.91      0.82        33

    accuracy                           0.79        61
   macro avg       0.80      0.78      0.78        61
weighted avg       0.80      0.79      0.78        61
```

Now this is the most instructive moment of the whole project — because your CV ranking and your test scores **disagree**, and learning to handle that is exactly the PhD-level judgment you're after. Let me lay out all the tuned models side by side:

Model	CV acc (reliable)	Test acc (noisy)	Class-1 recall	Class-1 F1
LogReg (tuned)	**0.8299**	0.7869	0.91	0.82
RF (RandomizedSearch)	0.8139	0.8197	—	—
RF (GridSearch)	0.8221	0.8033	0.97	0.84

Look at what happened: by **CV** (the trustworthy metric averaged over 5 folds), LogReg is on top — `0.8299` — just as the comparison chart predicted. But on the single **test split**, RF scored higher. The two rankings flip.

Which do you believe? **The CV ranking.** Remember why: the test set is 61 patients, so `0.7869` vs `0.8033` is a difference of **one single patient** (48 vs 49 correct). That's pure noise — you should never overturn a stable 5-fold CV result because of a one-patient swing on a tiny test set. So your original instinct stands: **LogReg is the best model here.**

Two honest caveats, because this is real reasoning, not a formula:

- If the *clinical* priority were "miss as few sick patients as possible" (maximize class-1 recall), RF's 0.97 vs LogReg's 0.91 is a legitimate reason to prefer RF — at the cost of more false alarms (lower precision). On small data like this, that's a defensible alternative, not a wrong answer.
- LogReg picked `C=0.01` — quite strong regularization, i.e. a deliberately simple model. That it still leads on CV tells you the signal in this data is mostly linear, which is itself a useful finding.

For the final champion I'd take **LogReg**: best CV, simplest, fastest, and — the real bonus — **interpretable**. Let's cash in that interpretability, then save it.

10. Final Model — Interpret, Save & Predict

LogisticRegression is the champion: best CV score, simplest, fastest, and — the real bonus — **interpretable**. Each coefficient is a log-odds weight; positive pushes toward disease, negative toward healthy. Compare the signs to the Section 2 correlations as a sanity check.

Then we **save the whole pipeline** (scaler + model) with `joblib` so new data gets identical preprocessing, **reload**

it, and **predict on a brand-new patient** — the complete deployable lifecycle.

In [57]:
```
## Chunk 49 — read the LogReg coefficients (the interpretability payoff)
best_lr = gs_lr.best_estimator_.named_steps["model"]   # pull model out of the pipeline
coefs = pd.Series(best_lr.coef_[0], index=X.columns).sort_values()
coefs

# A positive coefficient pushes toward disease (class 1), negative pushes toward healthy.
# Compare the signs to the correlations you computed in Chunk 9 — cp, thalach positive; ca, oldpeak
# When the model's coefficients agree with the raw stats, that's strong evidence it learned real bi
# (This is something RF's feature importances can't give you — they show which features matter but
```

Out[57]:
```
exang      -0.217101
oldpeak    -0.209106
thal       -0.207127
ca         -0.184061
sex        -0.155812
chol       -0.078399
age        -0.072478
trestbps   -0.057148
fbs        -0.010103
restecg     0.069241
slope       0.168685
thalach     0.187271
cp          0.272284
dtype: float64
```

In [58]:
```
# Chunk 50 — save the champion (the whole pipeline)
import joblib
joblib.dump(gs_lr.best_estimator_, "heart_disease_logreg.joblib")

# We save best_estimator_ — the full pipeline including the scaler — so new data gets the exact sam
```

Out[58]:
```
['heart_disease_logreg.joblib']
```

In [60]:
```
# Chunk 51 — reload and score a brand-new patient
loaded = joblib.load("heart_disease_logreg.joblib")
```

```
new_patient = pd.DataFrame([{
    "age": 54, "sex": 1, "cp": 0, "trestbps": 130, "chol": 250,
    "fbs": 0, "restecg": 1, "thalach": 150, "exang": 0, "oldpeak": 1.5,
    "slope": 2, "ca": 0, "thal": 2,
}])

print("Prediction:", loaded.predict(new_patient))            # 0 = no disease, 1 = disease
print("Probability:", loaded.predict_proba(new_patient).round(3))

# This completes the full lifecycle: the saved pipeline takes raw patient values, scales them, and
# confidence — exactly the pattern from your templates file.
```

```
Prediction: [1]
Probability: [[0.437 0.563]]
```

✅ Summary & Key Lessons

What we did: loaded and explored the data → split → compared 9 models → scaled the distance-based ones → cross-validated for a trustworthy ranking → tuned two models (RandomForest via Randomized+Grid search, LogisticRegression via GridSearch) → evaluated beyond accuracy → interpreted, saved, and deployed the champion.

Champion: tuned **LogisticRegression** (`C=0.01` , `l2`) — best CV accuracy, interpretable, strong recall for catching disease.

Lessons to carry forward:

1. **EDA first** — it dictates the whole strategy (missing values, target balance, scaling needs).
2. A "bad" model score often means **wrong preprocessing**, not a bad model (KNN/SVM + scaling).
3. **Trust cross-validation over a single test split**, especially on small data — rankings can flip, and CV is the reliable one.
4. **Match the metric to the goal** — accuracy hides error types; for disease, recall matters most.
5. **Tune the model that's actually winning**, and remember tuning has diminishing returns near the ceiling.
6. **Save the whole pipeline**, not just the model, so preprocessing travels with it.

Next project: repeat this exact playbook on a *messier* dataset (e.g. Titanic) with missing values and text columns, to practice building a real `ColumnTransformer`.